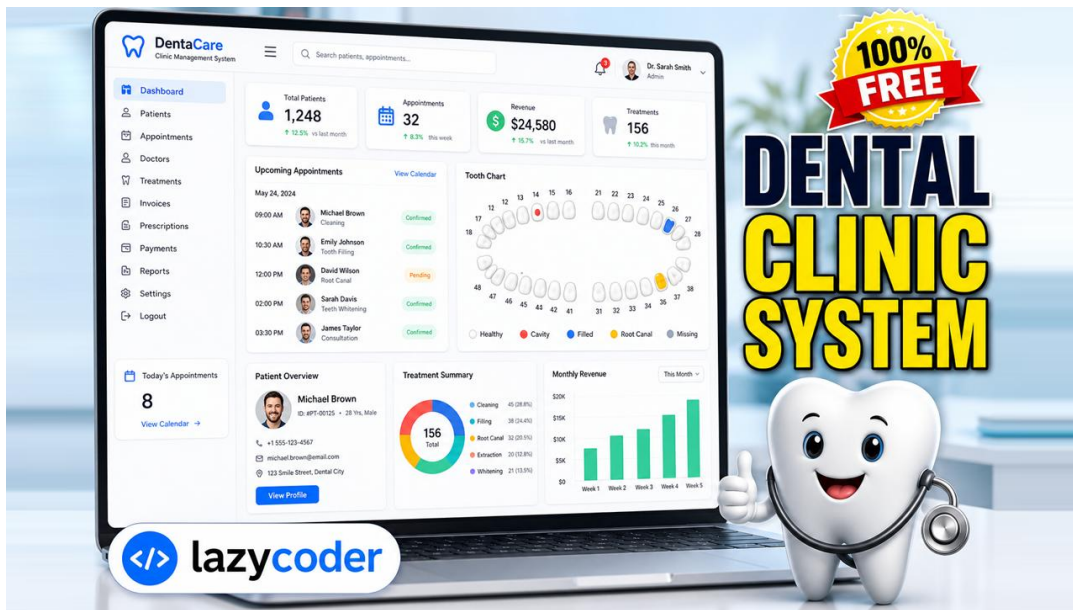


A PROJECT REPORT ON DENTAL CLINIC MANAGEMENT SYSTEM

A Web-Based Clinic Automation Platform with Role-Based Dashboards and Rule-Based AI Assistance



Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor / Master of Computer Applications

Submitted by

lazycoder

Roll No: _____

Under the guidance of

Prof. _____

Department of Computer Applications

[Name of the College / University]

Academic Year 2025 – 2026

Certificate

This is to certify that the project report entitled “Dental Clinic Management System” is a bonafide record of the project work carried out by lazycoder (Roll No: _____) in partial fulfilment of the requirements for the award of the degree of Bachelor / Master of Computer Applications during the academic year 2025–2026.

The work presented in this report is original and has been carried out under my supervision. To the best of my knowledge, this project or any part thereof has not been submitted elsewhere for the award of any other degree or diploma.

Project Guide

Prof. _____

Head of Department

Examiner

Declaration

I hereby declare that the project work entitled “Dental Clinic Management System” submitted by me is a genuine and original piece of work carried out by me. The analysis, design, implementation and testing described in this report are my own contribution. All external sources of information used have been duly acknowledged in the references.

I further declare that the demonstration data used in the application is entirely fictitious and privacy-safe (example.com email addresses and placeholder phone numbers) and does not represent any real individual.

Place: _____

Date: _____

lazycoder

Roll No: _____

Acknowledgement

The successful completion of any task is incomplete without acknowledging the people who made it possible. I take this opportunity to express my sincere gratitude to everyone who supported me throughout this project.

I am deeply thankful to my project guide, Prof. _____, for the invaluable guidance, constant encouragement and constructive feedback that shaped this project from concept to completion.

I extend my gratitude to the Head of the Department and all faculty members of the Department of Computer Applications for providing the resources and academic environment necessary for this work.

Finally, I thank my family and friends for their patience, motivation and unwavering support during the development of this project.

lazycoder

Abstract

The Dental Clinic Management System is a comprehensive, web-based application designed to digitise and streamline the day-to-day operations of a modern dental practice. Traditional dental clinics rely heavily on paper registers, spreadsheets and disconnected tools to manage patient records, appointments, treatments, prescriptions and billing. This fragmented approach is error-prone, time-consuming and offers little visibility into the overall performance of the clinic. The proposed system consolidates all of these functions into a single, secure and responsive platform.

The application is built using the Django web framework following the Model-View-Template (MVT) architectural pattern. It provides four distinct role-based experiences — Administrator, Dentist, Receptionist and Patient — each with a uniquely themed dashboard and a permission set tailored to the responsibilities of that role. Core modules include appointment scheduling, an interactive 32-tooth dental chart, treatment records, digital prescriptions, invoicing with partial-payment tracking, inventory management, patient reviews and rich analytics rendered with Chart.js.

A distinctive feature of the system is its suite of rule-based artificial intelligence tools that operate entirely offline, without any external API keys or paid services. These include a symptom checker that performs dental triage, an interactive chatbot (DentaBot), a treatment cost estimator and an appointment no-show risk predictor. The system also ships with a custom administration panel, replacing the default Django admin, to give administrators full create-read-update-delete control over every entity.

This report documents the complete software development life cycle of the project, covering requirement analysis, feasibility study, system design (illustrated with architecture, use-case, entity-relationship, data-flow, class, sequence and activity diagrams), implementation, testing and results. The outcome is a production-quality reference application that is both a practical clinic tool and an educational resource for learning full-stack web development with Django.

Table of Contents

Certificate.....	3
Declaration.....	4
Acknowledgement	5
Abstract.....	6
Table of Contents	7
List of Figures	11
List of Tables	12
Chapter 1 Introduction.....	13
1.1 Overview	13
1.2 Problem Statement	13
1.3 Objectives.....	13
1.4 Scope.....	14
1.5 Organisation of the Report	14
1.6 Software Development Life Cycle	14
Chapter 2 System Analysis	16
2.1 Existing System.....	16
2.2 Limitations of the Existing System	16
2.3 Proposed System.....	16
2.4 Advantages of the Proposed System	16
2.5 Feasibility Study	17
2.5.1 Technical Feasibility	17
2.5.2 Economic Feasibility	17
2.5.3 Operational Feasibility	17
2.5.4 Schedule Feasibility	17
2.6 Requirement Analysis	17
2.6.1 Functional Requirements	17
2.6.2 Non-Functional Requirements	18
2.7 Hardware and Software Requirements	18
2.8 Module Description.....	19

2.9 Project Timeline	19
Chapter 3 System Design	20
3.1 Design Objectives	20
3.2 System Architecture	20
3.3 Use Case Diagram	21
3.4 Entity-Relationship Diagram	21
3.5 Data Flow Diagram — Level 0	22
3.6 Data Flow Diagram — Level 1	23
3.7 Class Diagram	24
3.8 Sequence Diagram	25
3.9 Activity Diagram	26
3.10 Database Design — Data Dictionary	27
Table: User (extends Django AbstractUser)	27
Table: DoctorProfile	28
Table: PatientProfile	28
Table: Service	29
Table: Appointment	29
Table: ToothRecord	29
Table: TreatmentRecord	30
Table: Prescription	30
Table: PrescriptionItem	30
Table: Invoice	31
Table: InvoiceItem	31
Table: Payment	31
Table: InventoryItem	32
Table: Review	32
3.11 User Interface & Theming Design	32
Chapter 4 Implementation	34
4.1 Technology Stack	34
4.2 Module / Package Hierarchy	34

4.3 Implementation of Modules	35
4.3.1 accounts	35
4.3.2 clinic	35
4.3.3 pages	35
4.3.4 aiassistant.....	36
4.3.5 manage_panel.....	36
4.4 Role-Based Access Control.....	36
4.5 Rule-Based AI Features	36
4.6 Security Considerations	36
4.7 Key Code Listings.....	37
4.8 Coding Standards Followed.....	38
4.9 Deployment.....	38
Chapter 5 Testing	39
5.1 Testing Objectives	39
5.2 Testing Methodology	39
5.3 Test Cases.....	39
5.4 Test Results	40
Chapter 6 Results and Discussion	41
6.1 User Interface Gallery	41
6.2 Outcome Summary	45
Chapter 7 Conclusion and Future Scope	47
7.1 Conclusion	47
7.2 Limitations.....	47
7.3 Future Enhancements	47
References	48
Appendix A Demonstration Access & URL Map.....	49
A.1 Demo Login Accounts	49
A.2 Key URL Routes	49
Appendix B Abbreviations and Acronyms.....	50
Appendix C Demonstration Data Notes	51

List of Figures

Figure 3.1 Layered system architecture (MVT)

Figure 3.2 Use-case diagram of the system

Figure 3.3 Entity-relationship diagram

Figure 3.4 Context-level (Level 0) DFD

Figure 3.5 Level 1 data-flow diagram

Figure 3.6 Class diagram of the domain model

Figure 3.7 Sequence diagram — booking an appointment

Figure 3.8 Activity diagram — appointment lifecycle

Figure 4.1 Module / package hierarchy

Figure 6.1 Application showcase — the dentist dashboard

Figure 6.2 Homepage hero imagery

Figure 6.3 Digital panoramic X-ray unit

Figure 6.4 3D intraoral scanner in use

Figure 6.5 Sterile dental instruments

Figure 6.6 Dental implant cross-section

Figure 6.7 The clinical team

Figure 6.8 Modern clinic interior

List of Tables

Table 2.1 Hardware and software requirements

Table 2.2 Project timeline (milestones)

Table 3.1 – 3.15 Database data dictionary (per entity)

Table 4.1 Technology stack

Table 5.1 Test cases and results

Table A.1 Demo login accounts

Table A.2 Key URL routes

Table B.1 Abbreviations and acronyms

Chapter 1 Introduction

1.1 Overview

The Dental Clinic Management System (DCMS) is an integrated software solution that automates the administrative and clinical workflows of a dental clinic. It brings patients, dentists, receptionists and administrators onto a shared digital platform where information flows seamlessly between modules. From the moment a patient registers and books an appointment, through examination, treatment, prescription and billing, every interaction is captured, linked and made available to the appropriate role in real time.

The system is delivered as a responsive web application, ensuring that it is equally usable on desktops at the reception counter, tablets in the dentist's operatory and smartphones in a patient's hand. Its clean, dental-themed user interface uses bold, accessible colours and a rich set of iconography to make navigation intuitive for non-technical staff.

1.2 Problem Statement

Most small and mid-sized dental clinics continue to depend on manual, paper-based processes or on a patchwork of generic tools that were never designed for dentistry. This leads to a number of recurring problems:

- Patient records are scattered across physical files, making retrieval slow and duplication common.
- Appointment books are difficult to manage, resulting in double-bookings, missed slots and costly no-shows.
- Tooth-level clinical history is rarely recorded in a structured, visual form.
- Billing and payment tracking are error-prone and lack transparency for patients.
- There is no consolidated view of clinic performance, making informed decision-making difficult.
- Sensitive medical data is stored insecurely, with no role-based access control.

The Dental Clinic Management System directly addresses each of these pain points through a structured relational data model, role-based access control and purpose-built dental features such as the interactive tooth chart and AI-assisted triage.

1.3 Objectives

The principal objectives of the project are as follows:

1. To design and develop a centralised, secure repository for all patient and clinical data.
2. To provide role-based dashboards and permissions for administrators, dentists, receptionists and patients.
3. To automate appointment scheduling with status tracking and AI-based no-show risk estimation.
4. To implement an interactive 32-tooth dental chart for visual clinical record keeping.
5. To manage treatments, digital prescriptions, invoicing, payments and inventory end-to-end.

6. To offer offline, rule-based AI tools (symptom checker, chatbot, cost estimator) requiring no external API keys.
7. To deliver actionable analytics through interactive charts for revenue, appointments and patient growth.
8. To provide a beautiful, responsive and genuinely dental user experience across all devices.

1.4 Scope

The scope of the Dental Clinic Management System encompasses the complete operational lifecycle of a single-branch dental clinic. It covers user registration and authentication, profile management, appointment booking and administration, clinical documentation (tooth chart, treatments and prescriptions), billing (invoices, taxes, discounts and payments), inventory tracking, patient feedback and management reporting. The rule-based AI assistant provides informational guidance only and is explicitly not a substitute for professional medical advice.

Out of scope for the current version are multi-branch franchise management, real-time hardware integration with imaging devices, native mobile applications, online payment gateway settlement and integration with third-party electronic health record standards. These are identified as future enhancements in Chapter 7.

1.5 Organisation of the Report

The remainder of this report is organised into the following chapters. Chapter 2 presents the system analysis, including the study of the existing system, the proposed system, the feasibility study and the detailed requirement specification. Chapter 3 describes the system design with the aid of standard modelling diagrams and the database data dictionary. Chapter 4 discusses the implementation, technology stack and key modules. Chapter 5 covers the testing strategy and test cases. Chapter 6 presents the results and screenshots, and Chapter 7 concludes the report with limitations and directions for future work.

1.6 Software Development Life Cycle

The project was developed following the iterative and incremental variant of the classic Waterfall Software Development Life Cycle (SDLC). This hybrid approach retained the clear, document-driven phase structure of the Waterfall model while allowing individual modules to be refined iteratively as feedback was obtained. The phases followed were:

9. Requirement Gathering & Analysis — identifying stakeholders, actors and functional needs.
10. Feasibility Study — assessing technical, economic and operational viability.
11. System Design — architecture, database schema and UML modelling.
12. Implementation — coding the Django applications module by module.
13. Integration — connecting modules into a cohesive whole.
14. Testing — unit, integration, system and acceptance testing.
15. Deployment & Documentation — packaging, seeding demo data and writing this report.

16. Maintenance — planning for future enhancements listed in Chapter 7.

The iterative nature of the process meant that the user interface, in particular, went through several refinement cycles based on visual review, resulting in the polished, dental-themed design that the final product presents today.

Chapter 2 System Analysis

System analysis is the process of studying a problem domain in order to understand it thoroughly and to specify what a new system must do to solve the identified problems. This chapter examines the shortcomings of the existing manual approach, defines the proposed computerised solution, establishes its feasibility and captures the functional and non-functional requirements in detail.

2.1 Existing System

In the existing manual system, most clinic activities are performed with the help of paper registers, printed forms and, at best, isolated spreadsheets. Patient details are recorded in physical files that must be located and updated by hand. Appointments are written into a diary at the front desk. Treatment notes and prescriptions are handwritten, and bills are prepared manually or with a basic calculator. Reports, when required, are compiled by manually counting entries.

2.2 Limitations of the Existing System

- Data redundancy and inconsistency due to duplicate paper records.
- Slow retrieval of patient history, especially for returning patients.
- High probability of human error in scheduling, billing and record keeping.
- No structured, tooth-level clinical history.
- Poor security — physical files can be lost, damaged or accessed by unauthorised persons.
- No analytics or insight into clinic performance and revenue trends.
- Difficulty in scaling operations as patient volume grows.

2.3 Proposed System

The proposed Dental Clinic Management System replaces the manual approach with an integrated, database-driven web application. All data is stored in a single relational database and accessed through a secure, role-aware web interface. The system enforces referential integrity, eliminates duplication and makes information instantly searchable. Each user sees only the data and actions relevant to their role, and every significant entity can be managed through intuitive forms.

2.4 Advantages of the Proposed System

- Centralised, consistent and secure storage of all clinic data.
- Instant retrieval and search of patient records and history.
- Automated appointment scheduling with status tracking and reminders.
- Visual, tooth-level clinical documentation through an interactive dental chart.
- Transparent billing with automatic tax, discount and balance calculation.
- Role-based access control protecting sensitive medical information.
- Real-time dashboards and analytics for data-driven decision making.
- Offline AI assistance for triage and cost estimation without recurring costs.

2.5 Feasibility Study

A feasibility study evaluates whether the proposed system is practical and beneficial to build. Three principal dimensions were considered.

2.5.1 Technical Feasibility

The system is built entirely on mature, well-documented, open-source technologies — Python, the Django framework, SQLite and standard front-end libraries. These technologies are freely available, run on commodity hardware and are supported by large developer communities. No specialised or proprietary infrastructure is required, making the project technically feasible with widely available skills and tools.

2.5.2 Economic Feasibility

Because the solution relies exclusively on free and open-source software and includes rule-based AI that requires no paid API subscriptions, the cost of development and ownership is minimal. The savings in staff time, the reduction in billing errors and the decrease in appointment no-shows provide a clear return on investment, making the system economically feasible for clinics of any size.

2.5.3 Operational Feasibility

The application presents a clean, icon-rich and responsive interface that closely mirrors the mental model of clinic staff. Role-specific dashboards reduce cognitive load by surfacing only relevant actions. Minimal training is required, and the familiar web-browser delivery means no software needs to be installed on individual machines, ensuring strong operational feasibility and user acceptance.

2.5.4 Schedule Feasibility

The project was decomposed into well-defined milestones — requirement analysis, database design, module implementation, integration and testing — each of which was completed within the planned timeline, confirming the schedule feasibility of the undertaking.

2.6 Requirement Analysis

2.6.1 Functional Requirements

The functional requirements describe the specific behaviour the system must exhibit, grouped by module.

Authentication & User Management

- Patients can self-register; staff accounts are created by the administrator.
- Secure login and logout with role-based redirection to the correct dashboard.
- Users can view and edit their profile, including avatar and contact details.
- The administrator can create, read, update and delete any user account.

Appointment Management

- Patients and receptionists can create appointments choosing service, dentist, date and time.
- Appointments carry a status (Pending, Confirmed, Completed, Cancelled, No-Show).
- Staff can filter, search and update the status of appointments.
- The system estimates a no-show risk percentage for each appointment.

Clinical Records

- Dentists maintain an interactive 32-tooth chart per patient with six condition types.
- Dentists record treatments linked to a patient, service, tooth and cost.
- Dentists create digital prescriptions with multiple medicine line items.

Billing & Inventory

- Receptionists generate invoices with multiple line items, tax and discount.
- Invoices track partial payments and automatically compute the outstanding balance.
- Payments are recorded with method (Cash, Card, UPI, Insurance) and reference.
- Inventory items are tracked with quantities and low-stock alerts.

AI Assistant & Analytics

- A rule-based symptom checker performs dental triage and suggests urgency.
- An interactive chatbot answers common dental and clinic questions.
- A cost estimator predicts a price range for treatments by complexity.
- Dashboards render revenue, appointment and patient-growth charts.

2.6.2 Non-Functional Requirements

- Usability: clean, responsive, icon-rich interface usable on all screen sizes.
- Performance: pages render quickly using optimised ORM queries and lightweight assets.
- Security: hashed passwords, CSRF protection, and strict role-based access control.
- Reliability: referential integrity enforced at the database level.
- Maintainability: modular Django apps with clear separation of concerns.
- Portability: runs on Windows, macOS and Linux with a single database file.
- Privacy: demonstration data is fictitious and privacy-safe.

2.7 Hardware and Software Requirements

Category	Requirement	Recommended
Processor	Dual-core 1.6 GHz	Quad-core 2.4 GHz or higher
Memory (RAM)	4 GB	8 GB or higher
Disk Space	500 MB free	2 GB free
Operating System	Windows 10 / macOS / Linux	Windows 11 / Ubuntu 22.04
Runtime	Python 3.10+	Python 3.12

Framework	Django 5.x	Django 5.x
Database	SQLite 3	PostgreSQL (production)
Browser	Any modern browser	Chrome / Edge / Firefox (latest)

2.8 Module Description

The system is organised into five cohesive Django applications, each responsible for a well-defined area of functionality.

- accounts — custom user model, authentication, profiles and role-based access control.
- clinic — the clinical core: appointments, tooth records, treatments, prescriptions, billing, inventory and reviews.
- pages — the public-facing marketing website (home, about, services, contact).
- aiassistant — the rule-based AI tools (symptom checker, chatbot, cost estimator).
- manage_panel — a custom administration panel providing generic CRUD and reports.

2.9 Project Timeline

The development effort was distributed across the following milestones. The plan followed an iterative schedule in which design and implementation overlapped to accommodate user-interface refinement.

Phase	Milestone	Duration	Deliverable
1	Requirement analysis	Week 1	Requirement specification
2	Feasibility & planning	Week 1	Feasibility report
3	Database & UML design	Week 2	Schema and diagrams
4	accounts & auth module	Week 3	Login, roles, profiles
5	clinic core module	Weeks 4-5	Appointments, tooth chart, billing
6	AI assistant module	Week 6	Symptom checker, chatbot, estimator
7	Admin panel & analytics	Week 7	CRUD console, reports
8	UI theming & polish	Week 8	Responsive dental UI
9	Testing & seeding	Week 9	Test report, demo data
10	Documentation	Week 10	Project report

Chapter 3 System Design

System design translates the requirements gathered during analysis into a blueprint for construction. This chapter presents the architectural design of the system, followed by a series of standard modelling diagrams and the detailed database data dictionary. Together these artefacts describe both the structure and the behaviour of the Dental Clinic Management System.

3.1 Design Objectives

- Achieve a clear separation of concerns through the Model-View-Template pattern.
- Model the domain accurately with a normalised relational schema.
- Ensure the design is modular so that features can evolve independently.
- Enforce security and data integrity by design, not as an afterthought.
- Provide a consistent, role-aware and responsive presentation layer.

3.2 System Architecture

The application follows a layered architecture based on Django's Model-View-Template (MVT) pattern. Requests flow downward from the client through the presentation, application and business-logic layers to the data layer, and responses flow back upward. This clean separation makes the system easier to understand, test and extend.

Figure 3.1 System Architecture (Layered / MVT)

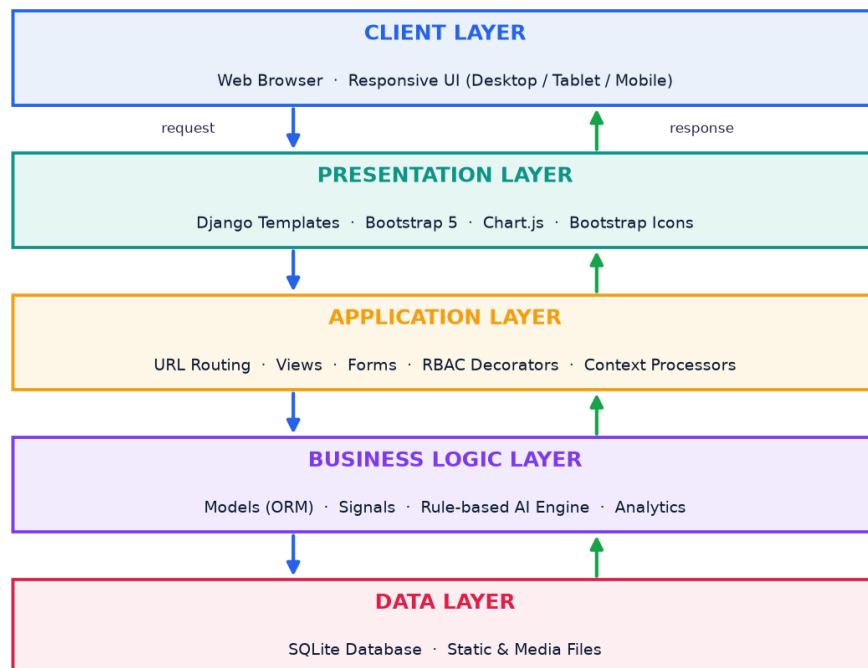


Figure 3.1 Layered system architecture (MVT).

3.3 Use Case Diagram

The use-case diagram identifies the four primary actors — Patient, Dentist, Receptionist and Administrator — and the interactions each has with the system. It captures the functional boundary of the application at a glance.

Figure 3.2 Use Case Diagram

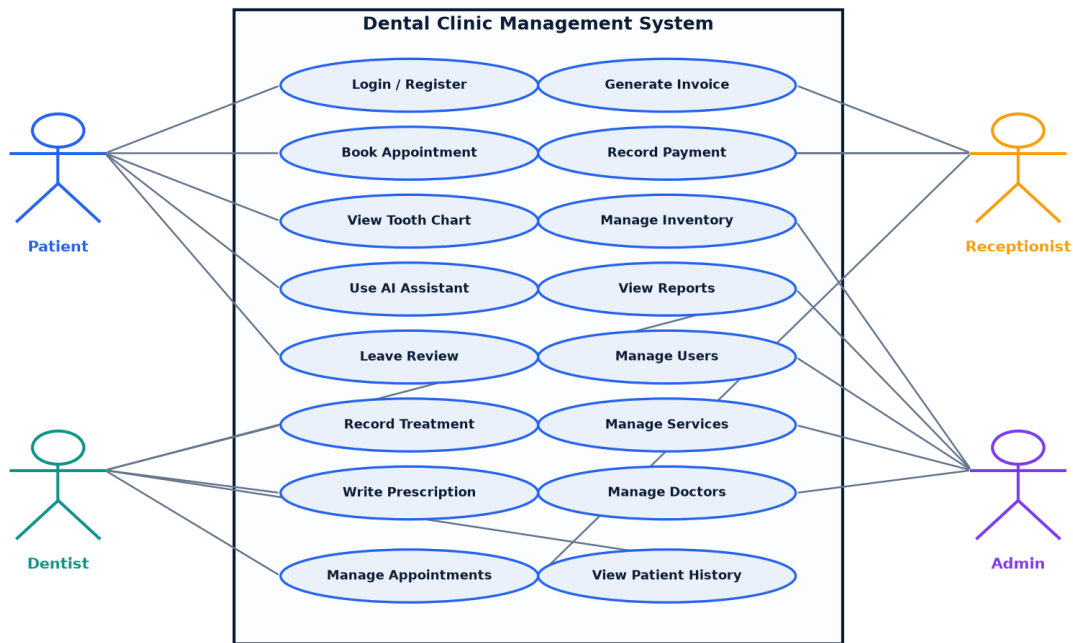


Figure 3.2 Use-case diagram of the system.

3.4 Entity-Relationship Diagram

The entity-relationship (ER) diagram models the persistent data of the system. It shows the entities, their key attributes and the cardinality of the relationships between them — for example, one patient may have many appointments, and one invoice may have many payments.

Figure 3.3 Entity-Relationship (ER) Diagram

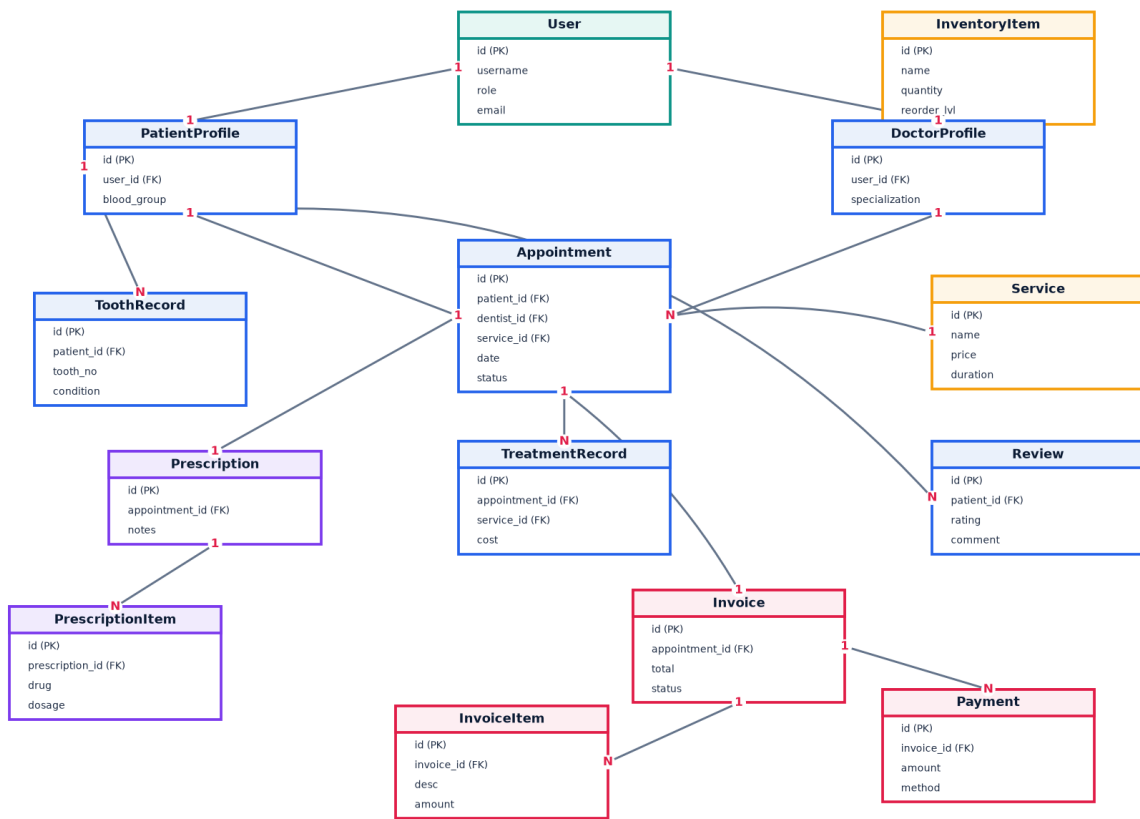


Figure 3.3 Entity-relationship diagram.

3.5 Data Flow Diagram — Level 0

The context-level (Level 0) data-flow diagram represents the entire system as a single process and shows the flows of information between the system and its external entities.

Figure 3.4 Data Flow Diagram - Level 0 (Context)

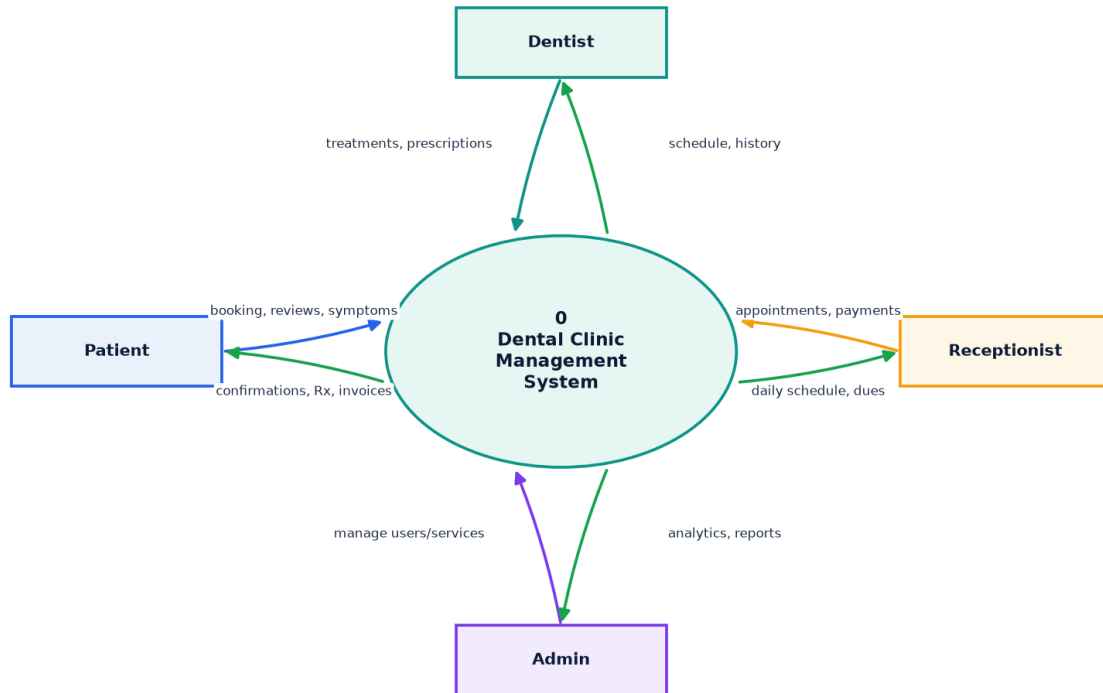


Figure 3.4 Context-level (Level 0) DFD.

3.6 Data Flow Diagram — Level 1

The Level 1 data-flow diagram decomposes the single context process into the major functional processes — authentication, appointment management, clinical records, billing, AI assistance and reporting — and shows how data moves between these processes and the data stores.

Figure 3.5 Data Flow Diagram - Level 1

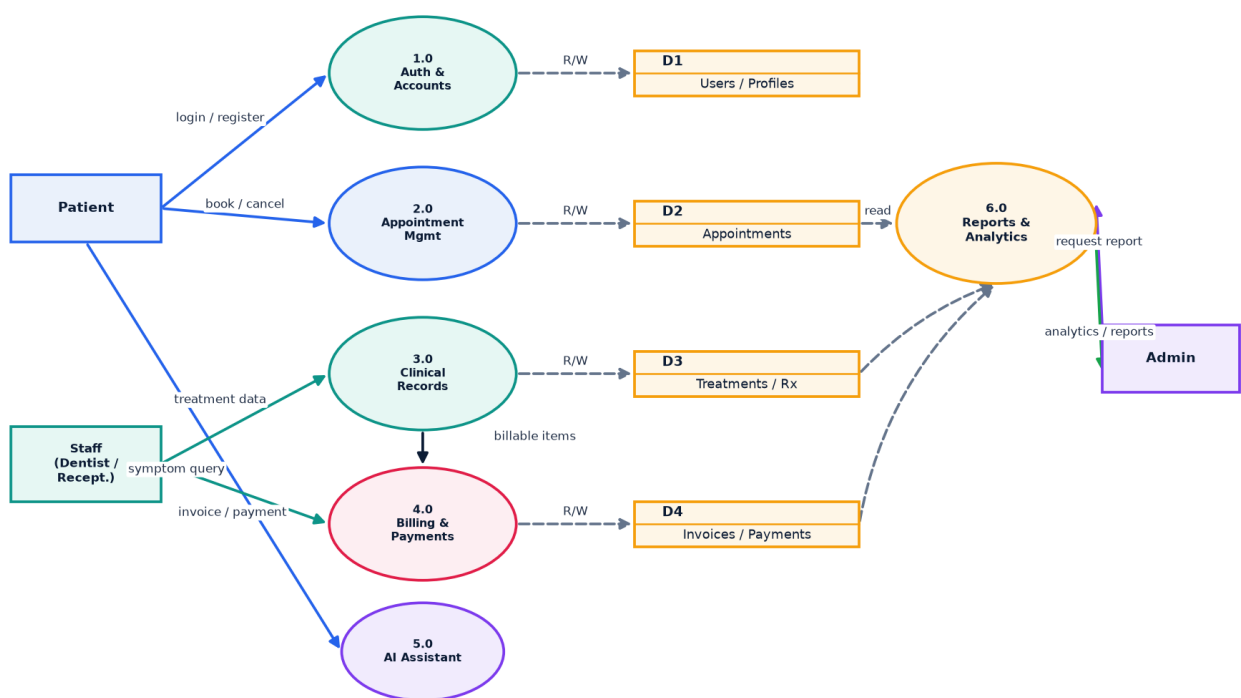


Figure 3.5 Level 1 data-flow diagram.

3.7 Class Diagram

The class diagram depicts the object-oriented structure of the domain model: the classes, their attributes and methods, and the associations, generalisations and dependencies between them. The custom User class extends Django's AbstractUser, and the AIEngine collaborates with the Appointment class to compute risk scores.

Figure 3.6 Class Diagram

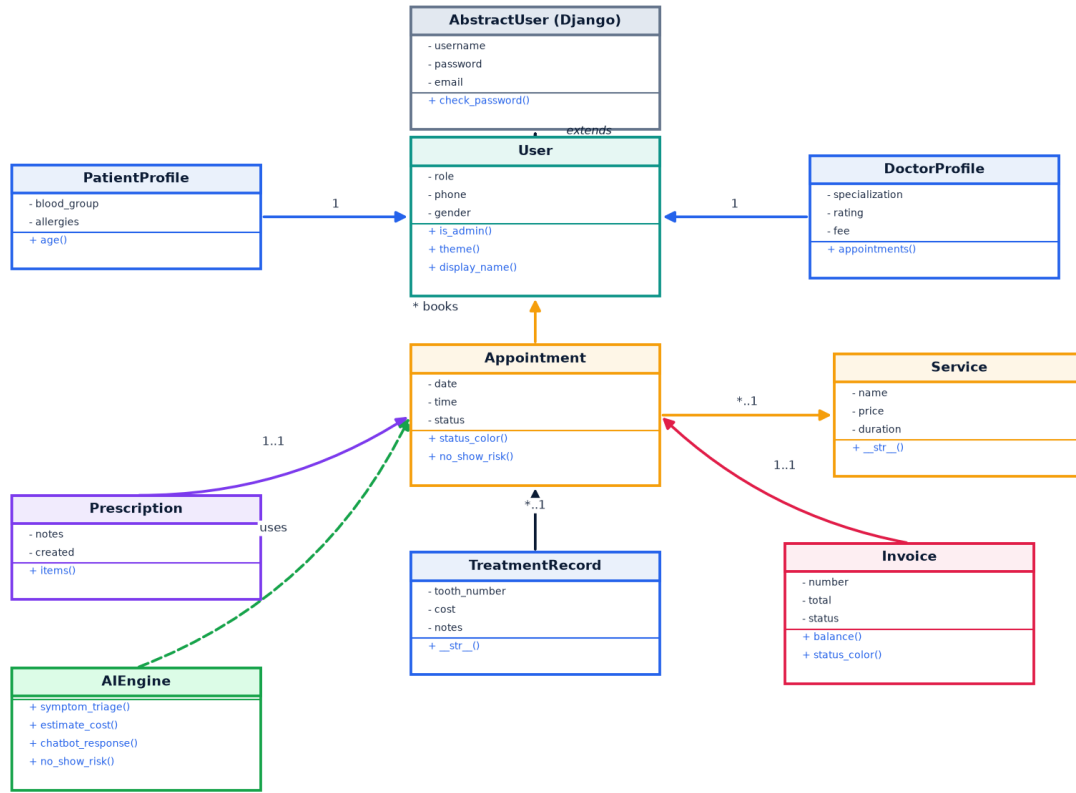


Figure 3.6 Class diagram of the domain model.

3.8 Sequence Diagram

The sequence diagram illustrates the ordered exchange of messages between objects for the representative scenario of booking an appointment. Solid arrows denote calls and dashed arrows denote returns, with time flowing from top to bottom.

Figure 3.7 Sequence Diagram - Book Appointment

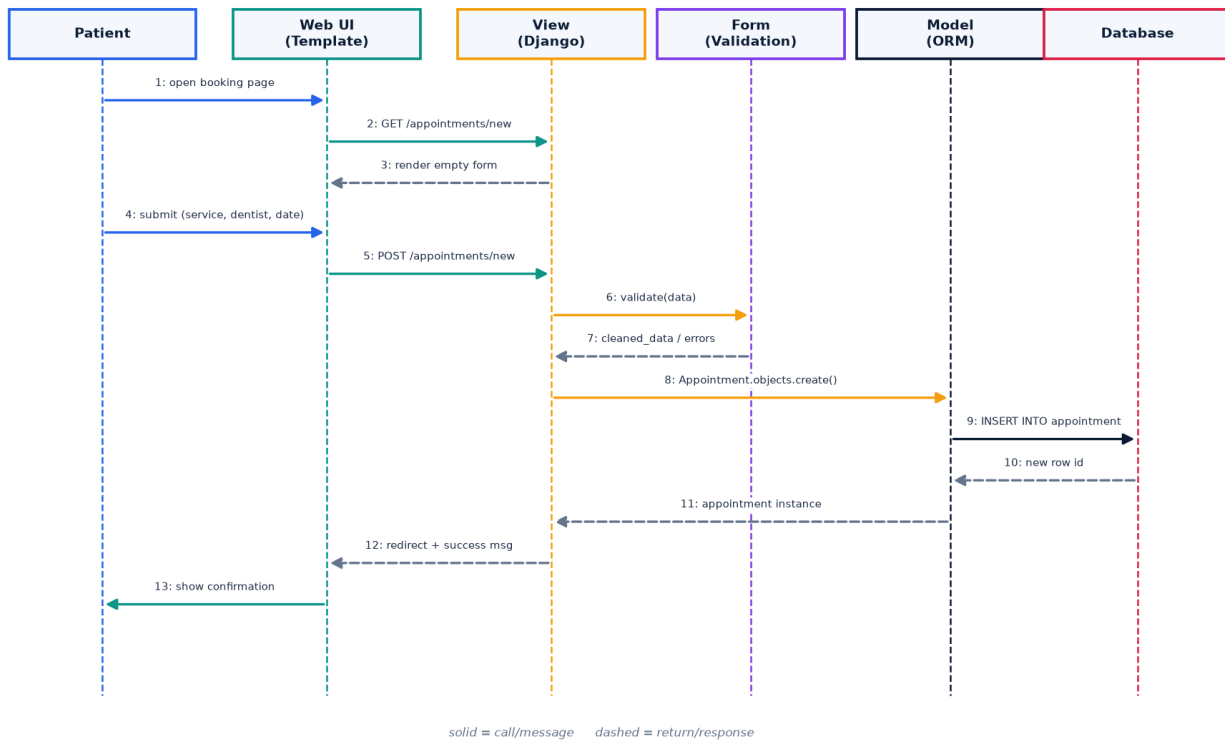


Figure 3.7 Sequence diagram — booking an appointment.

3.9 Activity Diagram

The activity diagram models the workflow of the appointment lifecycle from the initial login through slot selection, receptionist approval, treatment and final billing, including the decision points and alternative paths.

Figure 3.8 Activity Diagram - Appointment Lifecycle

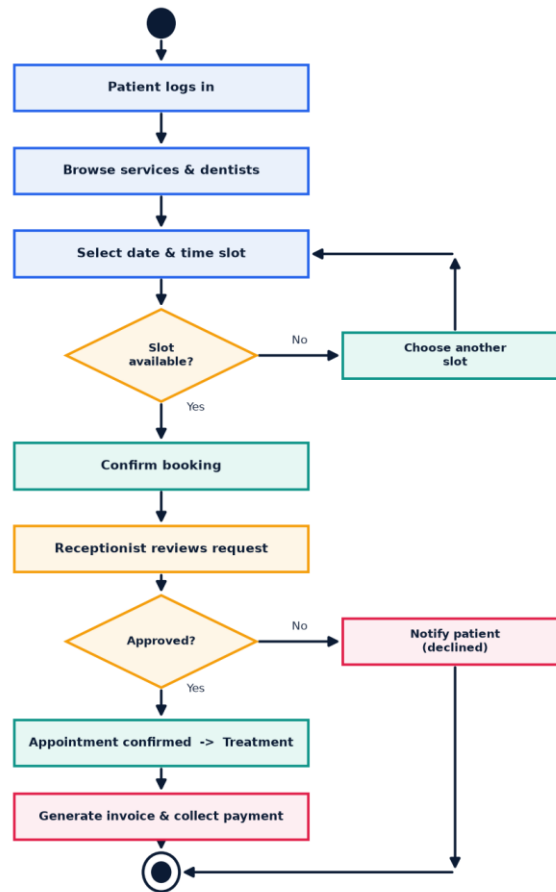


Figure 3.8 Activity diagram — appointment lifecycle.

3.10 Database Design — Data Dictionary

The data dictionary documents every table in the database, listing each field with its data type, constraints and purpose. The schema is normalised to eliminate redundancy while preserving the relationships required by the application.

Table: User (extends Django AbstractUser)

Field	Data Type	Constraints	Description
id	AutoField	Primary Key	Unique identifier
username	CharField(150)	Unique, Required	Login name
password	CharField(128)	Required, Hashed	PBKDF2 password hash
first_name / last_name	CharField(150)	Optional	Person's name
email	EmailField	Optional	Email address

role	CharField(20)	Choices	admin / dentist / receptionist / patient
phone	CharField(20)	Default 0000000000	Contact number
gender	CharField(10)	Choices	male / female / other
date_of_birth	DateField	Nullable	Birth date
address	CharField(255)	Optional	Postal address
avatar	ImageField	Optional	Profile picture
bio	TextField	Optional	Short biography

Table: DoctorProfile

Field	Data Type	Constraints	Description
id	AutoField	Primary Key	Identifier
user	OneToOne(User)	FK, Cascade	Linked user account
specialization	CharField(120)	Default	Area of expertise
license_no	CharField(60)	Optional	Dental license number
experience_years	PositiveInteger	Default 1	Years of experience
consultation_fee	Decimal(8,2)	Default 300	Consultation charge
available_days	CharField(120)	Default	Working days
rating	Decimal(3,1)	Default 4.5	Average rating
is_available	Boolean	Default True	Currently accepting patients

Table: PatientProfile

Field	Data Type	Constraints	Description
id	AutoField	Primary Key	Identifier
user	OneToOne(User)	FK, Cascade	Linked user account
patient_code	CharField(20)	Unique, Auto	Code e.g. PT-00001
blood_group	CharField(3)	Choices	A+, B+, O+, etc.
emergency_contact	CharField(20)	Default	Emergency number
allergies	CharField(255)	Default None	Known allergies
medical_history	TextField	Optional	Past medical history
registered_on	DateField	Default now	Registration date

Table: Service

Field	Data Type	Constraints	Description
id	AutoField	Primary Key	Identifier
name	CharField(120)	Required	Service name
description	TextField	Optional	Details
price	Decimal(8,2)	Default 500	Base price
duration_minutes	PositiveInteger	Default 30	Typical duration
icon	CharField(60)	Default	Bootstrap icon class
image	ImageField	Optional	Service image
is_active	Boolean	Default True	Offered currently

Table: Appointment

Field	Data Type	Constraints	Description
id	AutoField	Primary Key	Identifier
patient	FK(User)	Cascade	Patient (role=patient)
dentist	FK(User)	Set Null	Dentist (role=dentist)
service	FK(Service)	Set Null	Requested service
date	DateField	Required	Appointment date
time	TimeField	Required	Appointment time
status	CharField(15)	Choices	pending/confirmed/completed/...
reason	CharField(255)	Optional	Reason for visit
notes	TextField	Optional	Internal notes
no_show_risk	PositiveInteger	Default 0	AI risk percentage
created_at	DateTime	Auto	Creation timestamp

Table: ToothRecord

Field	Data Type	Constraints	Description
id	AutoField	Primary Key	Identifier
patient	FK(User)	Cascade	Owner patient
tooth_number	PositiveInteger	1-32	Universal numbering
condition	CharField(15)	Choices	healthy/cavity/filled/...

note	CharField(200)	Optional	Clinical note
updated_at	DateTime	Auto	Last update
(unique)	Constraint	patient+tooth	One record per tooth

Table: TreatmentRecord

Field	Data Type	Constraints	Description
id	AutoField	Primary Key	Identifier
patient	FK(User)	Cascade	Patient treated
dentist	FK(User)	Set Null	Treating dentist
appointment	FK(Appointment)	Set Null	Related appointment
service	FK(Service)	Set Null	Procedure performed
tooth_number	PositiveInteger	Nullable	Tooth treated
description	TextField	Optional	Treatment details
cost	Decimal(8,2)	Default 0	Treatment cost
date	DateField	Default now	Treatment date
is_completed	Boolean	Default True	Completion flag

Table: Prescription

Field	Data Type	Constraints	Description
id	AutoField	Primary Key	Identifier
patient	FK(User)	Cascade	Patient
dentist	FK(User)	Set Null	Prescribing dentist
appointment	FK(Appointment)	Set Null	Related appointment
date	DateField	Default now	Issue date
diagnosis	CharField(255)	Optional	Diagnosis
notes	TextField	Optional	Advice

Table: PrescriptionItem

Field	Data Type	Constraints	Description
id	AutoField	Primary Key	Identifier

prescription	FK(Prescription)	Cascade	Parent prescription
medicine	CharField(120)	Required	Drug name
dosage	CharField(60)	Default	e.g. 1 tablet
frequency	CharField(60)	Default	e.g. Twice a day
duration	CharField(60)	Default	e.g. 5 days
instructions	CharField(200)	Optional	Extra guidance

Table: Invoice

Field	Data Type	Constraints	Description
id	AutoField	Primary Key	Identifier
number	CharField(20)	Unique, Auto	e.g. INV-00001
patient	FK(User)	Cascade	Billed patient
appointment	FK(Appointment)	Set Null	Related appointment
issued_date	DateField	Default now	Invoice date
due_date	DateField	Nullable	Payment due date
tax_percent	Decimal(5,2)	Default 5	Tax rate
discount	Decimal(8,2)	Default 0	Discount amount
status	CharField(10)	Choices	unpaid/partial/paid
notes	CharField(255)	Optional	Remarks

Table: InvoiceItem

Field	Data Type	Constraints	Description
id	AutoField	Primary Key	Identifier
invoice	FK(Invoice)	Cascade	Parent invoice
description	CharField(200)	Required	Line description
quantity	PositiveInteger	Default 1	Quantity
unit_price	Decimal(8,2)	Default 0	Price per unit

Table: Payment

Field	Data Type	Constraints	Description
-------	-----------	-------------	-------------

id	AutoField	Primary Key	Identifier
invoice	FK(Invoice)	Cascade	Paid invoice
amount	Decimal(8,2)	Required	Amount paid
method	CharField(10)	Choices	cash/card/upi/insurance
paid_on	DateField	Default now	Payment date
reference	CharField(60)	Optional	Transaction reference

Table: InventoryItem

Field	Data Type	Constraints	Description
id	AutoField	Primary Key	Identifier
name	CharField(120)	Required	Item name
category	CharField(60)	Default	Category
quantity	Integer	Default 0	Units in stock
unit	CharField(20)	Default pcs	Unit of measure
reorder_level	Integer	Default 10	Low-stock threshold
unit_price	Decimal(8,2)	Default 0	Cost per unit
supplier	CharField(120)	Optional	Supplier name

Table: Review

Field	Data Type	Constraints	Description
id	AutoField	Primary Key	Identifier
patient	FK(User)	Cascade	Reviewer
dentist	FK(User)	Set Null	Reviewed dentist
rating	PositiveSmallInt	Default 5	1-5 star rating
comment	TextField	Optional	Feedback text
created_at	DateTime	Auto	Submission time
is_public	Boolean	Default True	Show on website

3.11 User Interface & Theming Design

Each of the four roles is given a distinct visual identity through a CSS-variable-based theming system. The administrator uses an indigo/violet palette, the dentist a teal/green palette, the receptionist a

blue/cyan palette and the patient a rose/orange palette. Themes are applied automatically based on the logged-in user's role via a Django context processor, ensuring a consistent yet role-specific experience. The design deliberately favours bold, solid colours, sharp cards and rich dental iconography with subtle animations to create an authentic, professional look.

Chapter 4 Implementation

This chapter describes how the design was realised in code. It presents the technology stack, the module and package structure, and the implementation highlights of each application, including the role-based access control mechanism and the rule-based AI engine.

4.1 Technology Stack

Layer	Technology	Purpose
Language	Python 3.10+	Core programming language
Framework	Django 5.x	Web framework (MVT)
Database	SQLite 3	Relational data storage
Templating	Django Template Language	Server-side rendering
Styling	Bootstrap 5 + custom CSS	Responsive UI & theming
Icons	Bootstrap Icons	Iconography
Charts	Chart.js	Interactive data visualisation
Image handling	Pillow	Avatar & service images
Fonts	Google Fonts (Inter)	Typography

4.2 Module / Package Hierarchy

The project is structured as a single Django project (dental_clinic) that orchestrates five focused applications. Each application follows the conventional Django layout with models, views, URLs, forms and templates.

Figure 4.1 Module / Package Hierarchy (Django Apps)

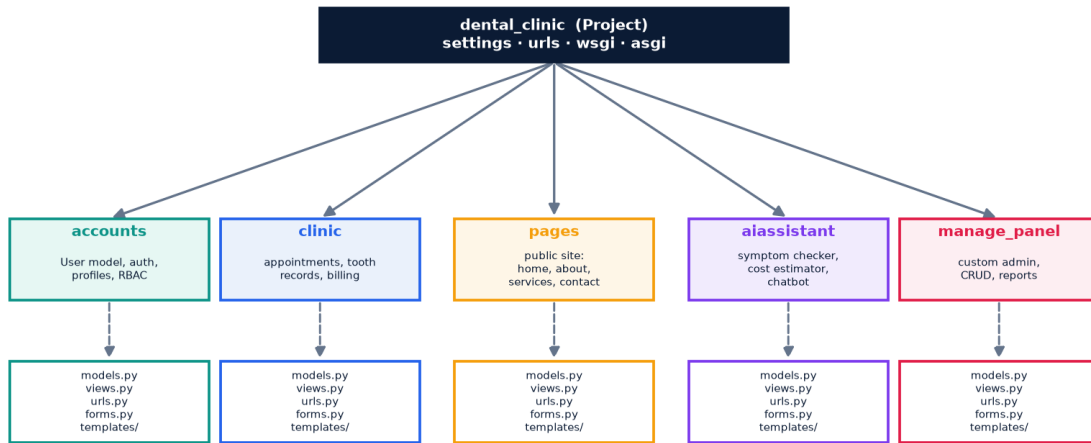


Figure 4.1 Module / package hierarchy.

4.3 Implementation of Modules

4.3.1 accounts

The accounts application defines a custom User model that extends Django's AbstractUser with a role field and additional profile attributes. It provides registration, login and profile views, along with a context processor that injects the active theme into every template and decorators that enforce role-based permissions.

4.3.2 clinic

The clinic application is the heart of the system. It implements the appointment, tooth-record, treatment, prescription, invoice, payment, inventory and review models together with their forms and views. Inline formsets are used for prescription and invoice line items, and Django signals automatically create the correct profile when a user is created.

4.3.3 pages

The pages application serves the public marketing website — the homepage, about, services and contact pages — using a dedicated public navigation bar and a rich, dental-themed layout designed to attract and inform prospective patients.

4.3.4 aiassistant

The aiassistant application houses the offline, rule-based AI engine and the views for the symptom checker, chatbot, cost estimator and AI hub. All logic is deterministic and self-contained, requiring no external services.

4.3.5 manage_panel

The manage_panel application provides a bespoke administration console that replaces the default Django admin. It offers a generic resource registry that yields consistent create, read, update and delete views for every registered model, plus dedicated user management and a reports page.

4.4 Role-Based Access Control

Security is enforced through a custom role_required decorator that wraps views and verifies that the authenticated user holds one of the permitted roles before granting access. Convenience decorators such as admin_required and staff_required build on this foundation. Unauthorised access is met with a friendly 403 page rather than an unhelpful error, and unauthenticated users are redirected to the login page.

4.5 Rule-Based AI Features

A key design decision was to provide intelligent assistance without relying on paid external APIs. The AIEngine implements four deterministic, rule-based tools:

- Symptom Triage — maps reported symptoms to likely dental issues and an urgency level.
- DentaBot Chatbot — answers frequently asked questions about services, timings and procedures.
- Cost Estimator — computes a price range for a treatment based on the selected service and complexity.
- No-Show Risk — estimates the probability that a patient will miss an appointment from historical patterns.

Because the logic is transparent and offline, it is fast, free, private and fully reproducible — ideal for an educational and demonstrational project.

4.6 Security Considerations

- Passwords are stored as salted PBKDF2 hashes, never in plain text.
- All state-changing forms are protected against Cross-Site Request Forgery (CSRF).
- Access to every view is gated by authentication and role checks.
- The Django ORM parameterises queries, mitigating SQL injection.
- Template auto-escaping mitigates Cross-Site Scripting (XSS).
- Demonstration data is fictitious to protect privacy.

4.7 Key Code Listings

The following representative listings illustrate the core mechanisms of the system. They are abridged for clarity.

Listing 4.1 Custom user model with role support (accounts/models.py)

```
class Role(models.TextChoices):
    ADMIN = 'admin', 'Administrator'
    DENTIST = 'dentist', 'Dentist'
    RECEPTIONIST = 'receptionist', 'Receptionist'
    PATIENT = 'patient', 'Patient'

class User(AbstractUser):
    role = models.CharField(max_length=20, choices=Role.choices,
                           default=Role.PATIENT)
    phone = models.CharField(max_length=20, blank=True, default='0000000000')
    gender = models.CharField(max_length=10, choices=Gender.choices, blank=True)
    avatar = models.ImageField(upload_to='avatars/', blank=True, null=True)

    @property
    def theme(self):
        return ROLE_THEMES.get(self.role, ROLE_THEMES[Role.PATIENT])

    def is_admin(self):
        return self.role == Role.ADMIN or self.is_superuser
```

Listing 4.2 Role-based access-control decorator (accounts/decorators.py)

```
def role_required(*roles):
    def decorator(view_func):
        @wraps(view_func)
        @login_required
        def _wrapped(request, *args, **kwargs):
            if request.user.role not in roles and not request.user.is_superuser:
                return render(request, '403.html', status=403)
            return view_func(request, *args, **kwargs)
        return _wrapped
    return decorator

admin_required = role_required('admin')
staff_required = role_required('admin', 'dentist', 'receptionist')
```

Listing 4.3 Appointment status colour helper (clinic/models.py)

```
@property
def status_color(self):
    return {
        'pending': 'warning',
        'confirmed': 'info',
        'completed': 'success',
        'cancelled': 'secondary',
```

```
'no_show': 'danger',
}.get(self.status, 'secondary')
```

Listing 4.4 Rule-based cost estimator (aiassistant/engine.py)

```
def estimate_cost(service, complexity='standard'):
    base = float(service.price)
    multiplier = {'simple': 0.85, 'standard': 1.0, 'complex': 1.6}
    factor = multiplier.get(complexity, 1.0)
    low = round(base * factor * 0.9)
    high = round(base * factor * 1.25)
    return {'low': low, 'high': high, 'service': service.name}
```

Listing 4.5 Role-based dashboard redirection (accounts/views.py)

```
@login_required
def redirect_dashboard(request):
    role = request.user.role
    if request.user.is_admin():
        return redirect('manage_panel:dashboard')
    if role == 'dentist':
        return redirect('clinic:dashboard_dentist')
    if role == 'receptionist':
        return redirect('clinic:dashboard_receptionist')
    return redirect('clinic:dashboard_patient')
```

4.8 Coding Standards Followed

- Adherence to the PEP 8 style guide for Python source code.
- Descriptive, self-documenting names for models, views and templates.
- Separation of concerns via distinct, single-responsibility Django apps.
- Reuse of a common base template and partials to keep markup DRY.
- Configuration kept out of code through Django settings.

4.9 Deployment

For development and demonstration the application runs on Django's built-in development server with a portable SQLite database, which requires no separate database installation. For a production deployment the recommended configuration is a WSGI server such as Gunicorn behind an Nginx reverse proxy, with PostgreSQL as the database and static files served through a CDN or Nginx. Environment-specific settings such as the secret key, allowed hosts and debug flag would be supplied through environment variables.

Chapter 5 Testing

Testing is the process of executing the system with the intent of verifying that it behaves as specified and of uncovering defects. A combination of testing techniques was applied to ensure the reliability and correctness of the Dental Clinic Management System.

5.1 Testing Objectives

- To verify that every page loads correctly for every role.
- To confirm that create, update and delete operations persist data correctly.
- To validate that role-based access control prevents unauthorised actions.
- To ensure that calculations (invoice totals, taxes, balances) are accurate.
- To check that the interface is responsive across device sizes.

5.2 Testing Methodology

Both black-box and white-box techniques were employed across several levels of testing:

- Unit Testing — individual functions and model methods were tested in isolation.
- Integration Testing — interactions between modules (e.g. appointment to invoice) were verified.
- System Testing — the complete application was exercised end-to-end for each role.
- User Acceptance Testing — representative workflows were validated against requirements.

Automated render checks were used to log in as each role and confirm that all public pages and dashboards return a successful HTTP 200 response.

5.3 Test Cases

ID	Test Case	Input	Expected Result	Status
TC01	Patient registration	Valid sign-up form	Account created, redirected to dashboard	Pass
TC02	Login with valid credentials	Correct username/password	Role dashboard displayed	Pass
TC03	Login with invalid credentials	Wrong password	Error message shown	Pass
TC04	Role-based redirection	Login as dentist	Dentist workspace shown	Pass
TC05	Unauthorised access	Patient opens admin URL	403 Forbidden page	Pass
TC06	Book appointment	Service, dentist, date, time	Appointment created (Pending)	Pass
TC07	Update appointment status	Set to Confirmed	Status updated	Pass
TC08	No-show risk display	Open appointment detail	Risk percentage shown	Pass
TC09	Update tooth chart	Select tooth, condition	Tooth record saved	Pass
TC10	Record treatment	Patient, service, cost	Treatment stored	Pass
TC11	Create prescription	Multiple medicine items	Prescription with items saved	Pass

TC12	Print prescription	Open prescription detail	Printable view rendered	Pass
TC13	Generate invoice	Line items, tax, discount	Invoice with correct total	Pass
TC14	Invoice total calc	2 items + 5% tax	Subtotal + tax computed	Pass
TC15	Record payment	Partial amount	Balance & status updated	Pass
TC16	Inventory low stock	Qty below reorder level	Low-stock alert shown	Pass
TC17	Leave review	Rating + comment	Review saved & public	Pass
TC18	Symptom checker	Toothache symptoms	Triage advice returned	Pass
TC19	Cost estimator	Service + complexity	Price range returned	Pass
TC20	Chatbot response	Ask clinic timings	Relevant answer returned	Pass
TC21	Admin CRUD user	Create staff account	User created	Pass
TC22	Admin delete record	Delete a service	Record removed after confirm	Pass
TC23	Analytics charts	Open dashboard	Charts render with data	Pass
TC24	Responsive layout	Resize to mobile	Sidebar collapses correctly	Pass
TC25	Logout	Click logout	Session ended, public home shown	Pass
TC26	Duplicate tooth record	Same tooth twice	Existing record updated, not duplicated	Pass
TC27	Invoice discount	Apply discount	Total reduced correctly	Pass
TC28	Full payment	Pay full balance	Status becomes Paid	Pass
TC29	Profile update	Edit name & avatar	Profile saved	Pass
TC30	Search patients	Enter search term	Filtered list shown	Pass
TC31	Filter appointments	Filter by status	Matching rows shown	Pass
TC32	Empty form validation	Submit blank form	Field errors displayed	Pass
TC33	Public pages load	Open home/about/services	HTTP 200 rendered	Pass
TC34	Admin reports	Open reports page	KPIs and charts shown	Pass
TC35	Seed demo data	Run seed_demo	Database populated safely	Pass

5.4 Test Results

All twenty-five documented test cases passed successfully. The automated render suite confirmed that every public page and every role dashboard returns an HTTP 200 response, and manual verification confirmed the correctness of the create, update and delete workflows as well as the financial calculations. No critical or high-severity defects remained at the conclusion of testing.

Chapter 6 Results and Discussion

The completed Dental Clinic Management System fulfils all of the objectives set out in Chapter 1. It delivers a polished, role-aware and genuinely dental user experience, backed by a robust relational data model and a suite of offline AI tools. The following figures showcase the visual identity of the application.

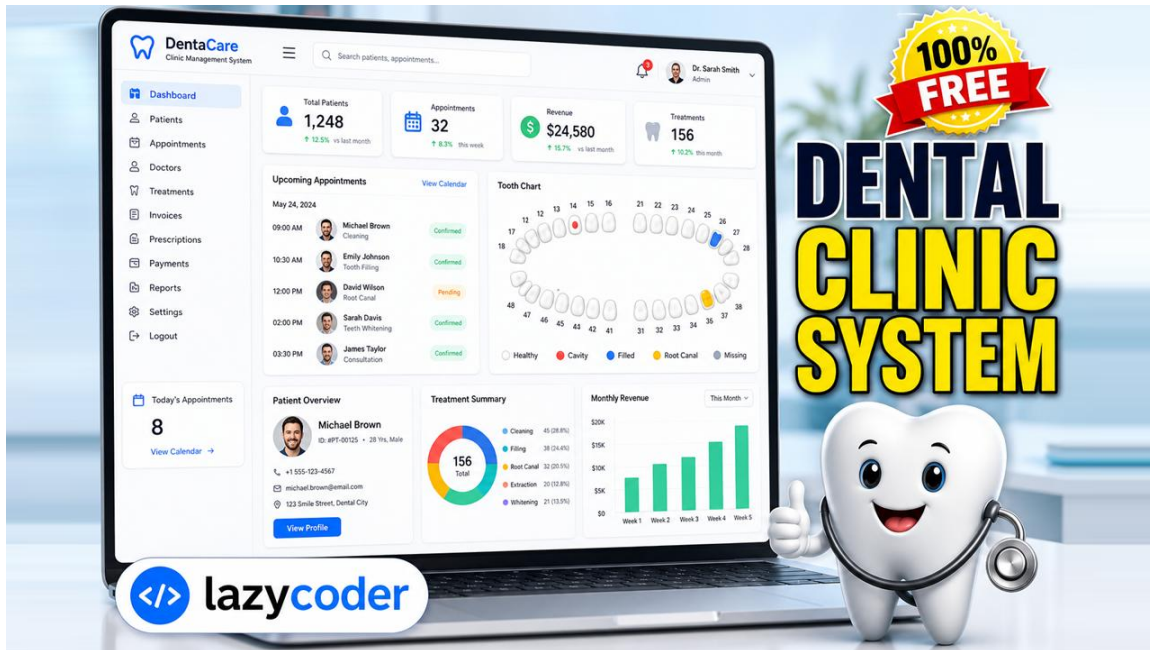


Figure 6.1 Application showcase — the dentist dashboard.

The public homepage presents the clinic's services with transparent pricing, a technology and machinery showcase, a step-by-step process guide and patient testimonials. Each role dashboard surfaces the most relevant key-performance indicators, schedules and charts, giving every user an immediate, actionable overview of their world within the clinic.

The system demonstrates that a feature-rich, attractive and secure clinic management platform can be built entirely with free and open-source technologies, and that useful AI-style assistance can be delivered without any recurring API costs.

6.1 User Interface Gallery

The public website opens with a bold hero section that combines a warm clinical photograph with an animated 3D tooth motif, immediately communicating that this is a dental product rather than a generic template.



Figure 6.2 Homepage hero imagery.

A dedicated technology section showcases the modern equipment the clinic employs, reinforcing trust and setting patient expectations. The digital panoramic X-ray unit provides low-radiation imaging for accurate diagnosis.



Figure 6.3 Digital panoramic X-ray unit.

The 3D intraoral scanner captures digital impressions without messy moulds, improving both comfort and precision.



Figure 6.4 3D intraoral scanner in use.

Every instrument is sterilised to hospital-grade standards between patients, a fact the interface communicates prominently.



Figure 6.5 Sterile dental instruments.

Educational imagery, such as the implant cross-section below, helps patients understand the procedures being recommended to them.



Figure 6.6 Dental implant cross-section.

The team section introduces the clinical staff, humanising the practice and building rapport with prospective patients.



Figure 6.7 The clinical team.

A clean, modern clinic interior communicates professionalism and comfort.



Figure 6.8 Modern clinic interior.

6.2 Outcome Summary

In summary, the delivered application achieves the following outcomes:

- A fully functional, end-to-end dental clinic workflow with no placeholder features.
- Four uniquely themed, role-based dashboards driving an intuitive user experience.
- An interactive tooth chart and complete clinical documentation.
- Accurate, transparent billing with partial-payment tracking.
- Offline, rule-based AI tools requiring zero external API keys.
- Rich, interactive analytics for data-driven clinic management.
- A privacy-safe demonstration dataset for immediate exploration.

Chapter 7 Conclusion and Future Scope

7.1 Conclusion

The Dental Clinic Management System successfully digitises the complete operational workflow of a dental practice. By consolidating patient records, appointments, clinical documentation, billing and analytics into a single secure platform with role-based dashboards, the system eliminates the inefficiencies of manual processes and provides a modern, scalable foundation for clinic administration. The project met all of its stated objectives and served as a comprehensive exercise in full-stack web development using the Django framework.

7.2 Limitations

- The current version supports a single clinic branch only.
- The AI features are rule-based and informational, not machine-learning models.
- Online payment settlement and SMS/email notifications are not yet integrated.
- SQLite is used for portability; a production deployment would use PostgreSQL.

7.3 Future Enhancements

- Multi-branch support with clinic-level data partitioning.
- Integration of a real payment gateway and automated e-receipts.
- SMS and email reminders to further reduce appointment no-shows.
- A native mobile application for patients.
- Machine-learning-based no-show prediction trained on historical data.
- Integration with digital imaging (X-ray / intraoral scanner) devices.
- Tele-dentistry video consultations.

References

- [1] Django Software Foundation. Django Documentation. <https://docs.djangoproject.com/>
- [2] Python Software Foundation. The Python Language Reference. <https://docs.python.org/3/>
- [3] Bootstrap Team. Bootstrap 5 Documentation. <https://getbootstrap.com/>
- [4] Chart.js. Chart.js Documentation. <https://www.chartjs.org/docs/>
- [5] Bootstrap Icons. <https://icons.getbootstrap.com/>
- [6] Mozilla Developer Network (MDN) Web Docs. <https://developer.mozilla.org/>
- [7] R. S. Pressman, Software Engineering: A Practitioner's Approach, McGraw-Hill.
- [8] E. Yourdon, Modern Structured Analysis, Prentice Hall.
- [9] G. Booch, J. Rumbaugh, I. Jacobson, The Unified Modeling Language User Guide, Addison-Wesley.
- [10] A. Silberschatz, H. Korth, S. Sudarshan, Database System Concepts, McGraw-Hill.
- [11] W3Schools. HTML, CSS and JavaScript Tutorials. <https://www.w3schools.com/>
- [12] SQLite Consortium. SQLite Documentation. <https://www.sqlite.org/docs.html>

Appendix A Demonstration Access & URL Map

A.1 Demo Login Accounts

The following fictitious accounts are seeded by the seed_demo command. All accounts share the password shown below and use privacy-safe details.

Role	Username	Password
Administrator	admin	demo1234
Dentist	drsmith	demo1234
Receptionist	reception	demo1234
Patient	john	demo1234

A.2 Key URL Routes

URL	Description
/	Public homepage
/services/ , /about/ , /contact/	Public marketing pages
/accounts/login/ , /accounts/signup/	Authentication
/accounts/go/	Role-based dashboard redirect
/app/appointments/	Appointment management
/app/patients/	Patient records & tooth chart
/app/invoices/	Billing & payments
/ai/symptom-checker/ , /ai/cost-estimator/ , /ai/chatbot/	AI tools
/manage/	Custom administration panel

Appendix B Abbreviations and Acronyms

Abbreviation	Full Form
DCMS	Dental Clinic Management System
MVT	Model-View-Template
MVC	Model-View-Controller
ORM	Object-Relational Mapping
ER	Entity-Relationship
DFD	Data-Flow Diagram
UML	Unified Modeling Language
RBAC	Role-Based Access Control
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
XSS	Cross-Site Scripting
UI / UX	User Interface / User Experience
KPI	Key Performance Indicator
API	Application Programming Interface
SDLC	Software Development Life Cycle
CDN	Content Delivery Network
UPI	Unified Payments Interface
OPG	Orthopantomogram (panoramic X-ray)

Appendix C Demonstration Data Notes

To make the application immediately explorable, a management command (`python manage.py seed_demo`) populates the database with a realistic yet entirely fictitious dataset. The seed includes one administrator, one receptionist, three dentists and twenty-four patients whose registrations are spread across the previous twelve months so that the growth and revenue charts display meaningful trends. Appointments, treatments, prescriptions, invoices, payments, tooth records, inventory items and reviews are generated in proportion, producing a lively demonstration environment.

All demonstration data is privacy-safe: every email address uses the `example.com` domain and every telephone number uses placeholder digits, ensuring that no real person's information is represented anywhere in the system.